

Tugas Pemrograman Berorientasi Objek

Kode Kelas	SI403
Dosen Pengajar	Dr. Fauziah , S.Kom., M.M.S.I.
Universitas	Universitas Siber Asia
Nama Mahasiswa	Angga Alfiansah
NIM	240101010032

Analisis Kode Program Java - Latihan Sesi 14 (Koneksi Database MySQL)

Analisis Koneksi Database MySQL pada Java

1. Pesan Error "Communications link failure"

- Penyebab:** Error ini berarti aplikasi Java gagal membangun jaring komunikasi dengan server MySQL. Penyebab utamanya bisa berupa: *Service MySQL* (seperti XAMPP) belum dinyalakan, port yang dituju salah atau diblokir oleh *firewall*, atau alamat *host/IP* tujuan tidak valid.
- Solusi:** Pastikan aplikasi server MySQL sudah berstatus *Running*. Periksa juga apakah port 3306 terbuka dan konfigurasi host URL (*localhost*) di source code sudah benar.

2. Tiga Faktor Kegagalan Koneksi (`DriverManager.getConnection`)

Jika koneksi database gagal dibuat pada sintaks kode di atas, analisis faktor penyebab utamanya adalah:

- Kredensial Salah:** *Username* "root" atau *password* "12345" tidak cocok dengan konfigurasi asli di server MySQL.
- Database Tidak Ditemukan:** Database dengan nama "akademik" belum dibuat di dalam server MySQL.
- Driver JDBC Belum Ada:** Library/Driver konektor MySQL (seperti `mysql-connector-java.jar`) belum ditambahkan ke *classpath/libraries* project Java Anda.

3. Data Tidak Tersimpan Setelah Tombol "Simpan" Ditekan

Bagian program yang harus diperiksa agar penyimpanan data berjalan dengan baik:

- Action Listener:** Pastikan tombol "Simpan" sudah memiliki *event listener* yang memanggil metode eksekusi ke database.
- Sintaks SQL Query:** Periksa apakah perintah `INSERT INTO ...` sudah benar penulisannya dan sesuai dengan jumlah maupun nama kolom di tabel database.
- Eksekusi Statement:** Pastikan kode memanggil fungsi `statement.executeUpdate()` setelah query disiapkan.
- Penanganan Error (Catch):** Periksa isi dari blok `catch` . Terkadang error tersembunyi karena blok `catch` dibiarkan kosong, sehingga seolah-olah sukses dieksekusi padahal ada data yang gagal disimpan karena ketidakcocokan tipe data.

4. Keunggulan `PreparedStatement` Dibandingkan `Statement`

Penggunaan `PreparedStatement` sangat disarankan karena:

- Keamanan (Anti SQL Injection):** `PreparedStatement` secara otomatis membersihkan (*escape*) karakter-karakter input berbahaya yang diberikan oleh pengguna, sehingga sangat aman dari peretasan injeksi SQL.
- Performa Lebih Baik:** Struktur query pada `PreparedStatement` dikompilasi (*pre-compiled*) oleh database. Jika dieksekusi berulang-kali (misal di dalam *looping* dengan parameter berbeda), prosesnya jauh lebih cepat dan tidak membebani database.

5. Error "Too many connections"

- Penyebab:** Aplikasi sering membuka koneksi database baru (misalnya setiap kali mengeklik tombol) namun **lupa/tidak pernah menutupnya** kembali. Akibatnya, batas maksimal koneksi simultan pada sistem MySQL penuh sehingga menolak koneksi baru.
- Cara Mengatasi:** Selalu pastikan untuk memanggil metode `.close()` pada objek `Connection` , `Statement` , dan `ResultSet` di dalam blok `finally` , atau gunakan sintaks `try-with-resources` (pada Java 7+) agar koneksi tertutup secara otomatis segera setelah penggunaannya selesai.

6. Aplikasi Berjalan di PC Pengembang, Namun Gagal di Komputer Lain

Faktor-faktor yang membedakan:

- Ketersediaan Database:** Struktur tabel database belum di-*export/import* (di-*dump*) ke komputer tujuan.
- Konfigurasi Jaringan (Localhost):** Jika database tetap menempel di PC pengembang, maka komputer klien tidak akan bisa mencarinya jika URL koneksi masih menggunakan `localhost` . Harus menggunakan IP Address PC pengembang, dan user MySQL pengembang harus memberikan hak akses eksternal (*remote access*).
- Library Tertinggal:** *Driver MySQL Connector* tidak ikut ter-*build* ke dalam file `.jar` atau foldernya tertinggal di PC tujuan.

7. Hardcode Username & Password di Source Code

- Dampak Keamanan:** Menulis *password* langsung ke teks program sangatlah rentan. Siapa saja yang mendapatkan *source code* (atau hanya dengan me-*reverse-engineer* file `.class` / `.jar`) bisa langsung melihat kata sandi server dan mencuri data.
- Solusi Lebih Baik:** Simpan *username* dan *password* di file konfigurasi eksternal (misalnya `.properties` atau `.env`). File ini dibaca saat *runtime* oleh Java dan dipastikan tidak ikut ter-upload ke repositori *version control* publik (di-*gitignore*).

8. Alasan Wajib Menggunakan `close()` atau `try-with-resources`

Koneksi jaringan ke sebuah database adalah *resource* yang memakan banyak memori, baik di sisi *client* Java maupun di server database itu sendiri. Jika tidak ditutup, akan terjadi **Resource Leak** (kebocoran sumber daya). Semakin lama aplikasi berjalan, "koneksi-koneksi hantu" ini akan menumpuk, menyebabkan memori tersita percuma, performa aplikasi melambat pesat, dan berujung pada aplikasi yang *crash* (mengalami *too many connections*).

9. Keuntungan DBMS dibandingkan Menyimpan Data di File Teks

- Integritas dan Relasi Data:** DBMS memiliki kemampuan menetapkan tipe data wajib dan menjalin relasi antar-tabel (*Primary/Foreign Key*) yang membuat struktur data sangat rapi dan mencegah duplikasi.
- Skalabilitas & Kecepatan Akses:** DBMS memakai arsitektur "Indeks". Mencari satu nama mahasiswa spesifik dari jutaan data bisa dilakukan dalam milidetik. Berbeda dengan file teks yang harus dibaca ulang baris demi baris dari awal hingga akhir.
- Konkurensi Sistem:** DBMS membolehkan puluhan pengguna melakukan `INSERT/UPDATE` data dalam waktu yang sama persis tanpa merusak format data (*data corruption*). File teks akan langsung rusak jika diedit dua program secara bersamaan.

10. Data di Tampilan GUI Java Tidak Sesuai (Stale) Setelah Operasi Update

- Kemungkinan Penyebab:** Logika kode SQL `UPDATE` sukses di database, tetapi **tampilan GUI (misal `JTable`) tidak disinkronisasi/di-*refresh***. Komponen GUI masih mengingat data lama yang tersimpan dalam *model* (memori) aplikasinya.

- **Langkah Mengatasi:** Panggil kembali sebuah fungsi khusus yang bertugas membaca data (`SELECT`) segera setelah fungsi eksekusi `UPDATE` tersebut berhasil. Kosongkan baris pada GUI tabel yang lama (contoh `tableModel1.setRowCount(0)`), lalu lemparkan hasil data `SELECT` yang terbaru ke model tersebut agar tampilannya tersinkron dan mutakhir.